

Characterizing GPU Data Compression Performance Across AMD Architectures

Cristiano Künas¹[0000–0003–1080–7230], Gabriel Freytag¹[0000–0001–6081–5904],
and Philippe Navaux¹[0000–0002–9957–5861]

Federal University of Rio Grande do Sul (UFRGS), Porto Alegre, Brazil
`{cakunas, gfreytag, navaux}@inf.ufrgs.br`

Abstract. GPU-accelerated data compression can mitigate I/O bottlenecks in high-performance computing, yet the performance characteristics of lossless compression on AMD GPUs remain largely unexplored. This paper presents a quantitative characterization of LZ4, Snappy, and Cascaded compression across three generations of AMD GPUs (MI50, MI210, MI300X) and the RDNA 3-based RX 7900 XT, using both synthetic and seismic simulation datasets. We also describe the porting of these algorithms from CUDA to HIP via the ROCm ecosystem. Our results reveal that the MI300X achieves up to **[TODO: XX]**× higher decompression throughput than the MI50, and that CDNA and RDNA architectures exhibit complementary strengths: RDNA 3 outperforms CDNA 3 for compression due to its cache hierarchy, while CDNA 3 dominates decompression through superior memory bandwidth. An execution time breakdown further shows that PCIe transfer overhead dominates end-to-end time, indicating that kernel-level gains must be paired with integration strategies such as compression of GPU-resident data. These results establish baseline performance metrics across AMD GPU generations and identify concrete optimization paths for future work.

Keywords: GPU Computing · Data Compression · LZ4 · AMD · HIP · ROCm · High-Performance Computing

1 Introduction

Modern scientific computing faces an increasingly critical challenge: the volume of data generated by simulations is growing faster than our ability to move it efficiently. High-performance computing (HPC) applications in seismic imaging, climate modeling, and computational fluid dynamics routinely produce datasets ranging from terabytes to petabytes, making data movement—rather than computation—the dominant bottleneck in many workflows. This I/O wall has motivated significant interest in data compression techniques that can reduce storage requirements and transfer times without sacrificing scientific fidelity.

GPU-accelerated compression has emerged as a particularly attractive solution, leveraging the massively parallel architecture of graphics processors to achieve throughputs that can exceed storage system bandwidths. On NVIDIA

platforms, the nvCOMP library [25] has become the de facto standard, offering highly optimized implementations of algorithms such as LZ4, Snappy, and Cascaded that achieve compression throughputs exceeding hundreds of GB/s on modern GPUs. This mature ecosystem has enabled researchers to seamlessly integrate high-performance compression into their CUDA-based workflows.

However, the HPC landscape is becoming increasingly heterogeneous. AMD GPUs now power several of the world’s most powerful supercomputers, including Frontier at Oak Ridge National Laboratory — currently the world’s fastest supercomputer — and LUMI at CSC Finland, which ranks among the top systems on the TOP500 list. This shift creates a pressing need for portable, high-performance software tools that can execute efficiently across different GPU vendors. Unfortunately, a significant **tool gap** exists: while the CUDA ecosystem benefits from years of mature compression libraries, AMD users lack equivalent tools. Existing community efforts provide basic functionality but have not received the same level of optimization, leaving researchers with AMD hardware at a disadvantage for data-intensive workflows.

This paper addresses this gap through two contributions. First, we present a **quantitative characterization** of compression performance across three generations of AMD GPUs, spanning the CDNA and RDNA architectures. Using both synthetic datasets and scientific data representative of seismic applications, we establish baseline performance metrics and analyze how compression throughput scales with architectural improvements. Second, we describe the **porting of these algorithms from CUDA to HIP**, bringing key lossless compression algorithms to AMD platforms. While this implementation prioritizes functional correctness over vendor-specific optimizations, it demonstrates achievable performance levels and provides a foundation for future optimization.

Our experimental evaluation spans four AMD GPUs: the MI50 (CDNA/Vega), MI210 (CDNA 2), MI300X (CDNA 3), and RX 7900 XT (RDNA 3). We evaluate three compression algorithms — LZ4, Snappy, and Cascaded — across datasets ranging from highly compressible (zeros) to realistic scientific data (TTI wavefields from seismic simulations). Our results reveal several key findings:

1. Decompression throughput scales significantly across AMD GPU generations, with MI300X achieving up to **[TODO: XX]×** higher throughput than the MI50 baseline.
2. CDNA and RDNA architectures exhibit complementary strengths: RDNA 3 outperforms CDNA 3 for compression workloads, while CDNA 3 dominates decompression due to its higher memory bandwidth.
3. PCIe transfer overhead dominates end-to-end execution time on all platforms, indicating that kernel-level gains alone yield diminishing returns without integration with GPU-resident data workflows.

The remainder of this paper is organized as follows. Section 2 provides background on GPU-based compression and motivates the need for AMD-compatible tools. Section 3 describes our porting approach and implementation. Section 4 details the experimental methodology. Section 5 presents our cross-generation

performance analysis. Section 6 discusses implications and future directions. Section 7 reviews related work, and Section 8 concludes the paper.

2 Background and Motivation

2.1 GPU-based Data Compression

In scientific computing, lossless compression is often preferred as it guarantees exact data reconstruction. Among lossless algorithms, LZ4 [11] offers a favorable balance between speed and ratio, Snappy [16] prioritizes minimal latency, and Cascaded compression applies multiple encoding stages (delta + RLE + bit-packing) for higher ratios on structured data. All three can be parallelized across GPU threads, with their implementation details described in Section 3.3.

NVIDIA’s nvCOMP library [25] provides optimized CUDA implementations of these and other algorithms, achieving throughputs that can saturate the memory bandwidth of high-end GPUs. Version 2.2, which serves as the codebase for our HIP port, remains widely deployed. However, nvCOMP is tightly coupled to the CUDA ecosystem, limiting its applicability to other platforms.

2.2 The AMD GPU Landscape

AMD has made substantial inroads in high-performance computing, with its GPUs now powering some of the world’s fastest supercomputers. This success is built on two distinct GPU architectures optimized for different markets.

CDNA (Compute DNA) targets datacenter and HPC workloads, emphasizing compute throughput, high-bandwidth memory (HBM), and features specifically designed for scientific computing, such as matrix cores for AI workloads. The evolution from CDNA (MI50/MI100) through CDNA 2 (MI210/MI250X) to CDNA 3 (MI300X) has brought substantial improvements in both raw performance and memory bandwidth. The MI300X, in particular, features 304 compute units and 192 GB of HBM3 memory with over 5 TB/s of bandwidth, making it well-suited for memory-intensive workloads like data compression.

RDNA (Radeon DNA) targets gaming and consumer applications but has found use in cost-sensitive HPC deployments. RDNA 3, exemplified by the RX 7900 XT, introduces a chiplet-based design with separate compute and memory dies, achieving competitive compute performance at lower power consumption. While not designed for datacenter use, RDNA GPUs offer an interesting point of comparison for understanding how architectural choices affect compression performance. Table 1 summarizes the key specifications of the GPUs evaluated in this work.

AMD’s software ecosystem centers on ROCm (Radeon Open Compute), an open-source platform that provides tools for GPU computing. The HIP (Heterogeneous Computing Interface for Portability) programming model enables developers to write code that targets both AMD and NVIDIA GPUs with minimal modifications, thereby facilitating portability. However, while HIP simplifies porting CUDA code, achieving optimal performance on AMD hardware often requires architecture-specific optimizations that go beyond simple API translation.

Table 1. AMD GPU specifications evaluated in this study.

GPU	Arch.	CUs	Memory	BW (GB/s)	TDP (W)
MI50	CDNA	60	16 GB HBM2	1,024	300
MI210	CDNA 2	104	64 GB HBM2e	1,638	300
MI300X	CDNA 3	304	192 GB HBM3	5,300	750
RX 7900 XT	RDNA 3	84	20 GB GDDR6	800	315

2.3 The Compression Tool Gap

Despite AMD’s growing presence in HPC, the availability of optimized GPU compression tools has not kept pace with the hardware. The CUDA ecosystem benefits from nvCOMP — a vendor-supported, production-ready library available since 2020, offering architecture-tuned implementations of LZ4, Snappy, Cascaded, zstd, Deflate, ANS, and others. In contrast, the HIP/ROCm ecosystem relies on community-driven efforts such as hipCOMP, which appeared in 2022 and remains experimental, with limited algorithm coverage (LZ4 and Snappy), direct-port optimization, and sparse documentation.

This asymmetry creates practical challenges for the growing number of HPC systems built on AMD GPUs. Scientists working on AMD-equipped systems — including users of Frontier, LUMI, and other major supercomputers — cannot directly benefit from the mature CUDA compression ecosystem and must either port code themselves, use suboptimal tools, or forgo GPU-accelerated compression entirely. This situation motivates our work: by characterizing the current state of compression on AMD GPUs and providing an open implementation, we aim to bridge this gap and contribute to a vendor-agnostic compression ecosystem for HPC.

3 GPU Compression Library for AMD Platforms

This section describes the implementation of a GPU-accelerated lossless compression library targeting AMD GPUs via the HIP programming model. Our goal was to create a functional, portable implementation that enables performance characterization across GPU architectures, prioritizing correctness and cross-platform compatibility over vendor-specific optimizations. We also describe the benchmark infrastructure and containerization strategy built to support reproducible experiments.

3.1 Porting Approach

Our implementation is based on the open-source nvCOMP 2.2 codebase [25], which provides a batched compression API that processes multiple independent data chunks in parallel — a design well-suited to GPU architectures. We

chose this codebase as a starting point because it contains well-structured, GPU-optimized implementations of the three target algorithms (LZ4, Snappy, Cascaded) and its batched API model maps naturally to the HIP programming model. Our porting methodology follows the standard CUDA-to-HIP translation workflow documented in the HIP programming guide [3] and involved three phases.

Phase 1: API translation. We used AMD’s `hipify-perl` tool [3] to perform the initial mechanical translation of CUDA source code to HIP. The tool handled approximately 90% of the API surface automatically; the remaining manual modifications involved replacing CUB [24] device primitives with hipCUB equivalents, adapting warp-level intrinsics, and resolving header conflicts between CUDA and HIP include paths.

Phase 2: Build system adaptation. Configuring the CMake build system proved to be the most time-consuming aspect of the porting. We restructured the build to detect the HIP compiler (`hipcc`) and target specific GPU architectures through the `CMAKE_HIP_ARCHITECTURES` variable. Each target architecture is specified by its GFX identifier (e.g., `gfx906` for MI50, `gfx90a` for MI210, `gfx942` for MI300X, `gfx1100` for RX 7900 XT). A critical build-time option, `USE_WARPSIZE_32`, controls whether the library compiles with 32-thread or 64-thread wavefront assumptions, as discussed in Section 3.2.

Phase 3: Correctness validation. We validated the port using a correctness check: for every test input, we verified that `decompress(compress(data)) == data` with bit-exact equality across all chunk sizes, data types, and algorithms. This round-trip validation is a standard technique for verifying lossless compression implementations and ensures that no data corruption was introduced during the porting process.

3.2 Wavefront Size Adaptation

A fundamental architectural difference between NVIDIA and AMD GPUs is the width of the SIMT execution unit. NVIDIA GPUs use 32-thread *warps*, while AMD CDNA GPUs (gfx9 family) default to 64-thread *wavefronts* [2]. RDNA architectures (gfx10 and above) use 32-thread wavefronts natively [3]. This difference has significant implications for compression kernels, where warp-level primitives (ballot, shuffle, reduction) are used extensively for intra-warp communication.

Rather than rewriting all warp-level logic — which would require substantial algorithmic changes — we adopted a compile-time adaptation strategy. The library defines a `warpsize` constant and a `lane_mask_t` type that are selected based on build configuration:

- **CDNA targets** (MI50, MI210, MI300X): compiled with `ENABLE_HIP_OPT_WARPSIZE64`, using 64-bit lane masks and `warpsize = 64`. This matches the hardware’s native wavefront width.
- **RDNA targets** (RX 7900 XT): compiled with `USE_WARPSIZE_32`, using 32-bit lane masks and `warpsize = 32`. RDNA 3 only supports wave32 mode.

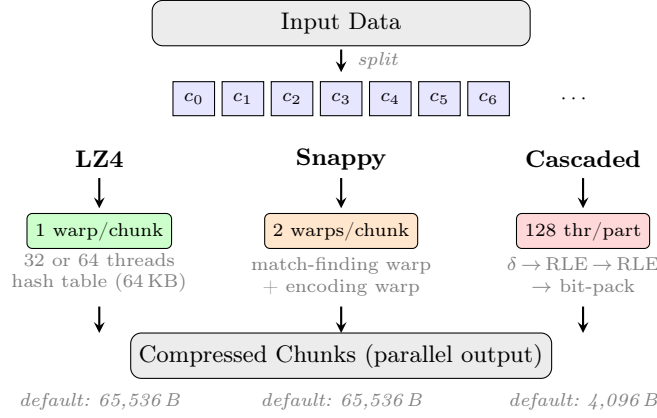


Fig. 1. Batched chunked compression model. Input data is split into independent chunks processed in parallel. Each algorithm assigns different thread configurations per chunk: LZ4 uses a single warp, Snappy uses two cooperating warps, and Cascaded uses 128 threads per 4 KB partition with a multi-stage encoding pipeline.

This approach preserves correctness across architectures while allowing selected kernels to exploit the wider wavefront when available. However, it does not address deeper optimization opportunities such as restructuring thread cooperation patterns to better match AMD’s memory hierarchy, which we leave for future work.

3.3 Chunked Compression Model

All three algorithms use a *batched chunked* compression model [25], where input data is divided into fixed-size chunks compressed independently in parallel (Figure 1). Each chunk is assigned to one or more thread blocks, enabling thousands to be processed concurrently. The algorithms differ in their thread-to-chunk mapping: **LZ4** assigns one warp per chunk with a private hash table (64 KB) for match-finding; **Snappy** uses two cooperating warps (match-finding + encoding); and **Cascaded** applies a multi-stage pipeline (delta + RLE + bit-packing) with 128 threads per 4 KB partition. These different parallelization strategies lead to distinct performance characteristics across architectures, as analyzed in Section 5.

3.4 Implementation Environment and Reproducibility

Reproducibility in GPU computing is challenging due to the tight coupling between application code, driver versions, and hardware [6,20]. To address this, we containerized our complete build and execution environment using Singularity [20], which is the standard container runtime in HPC environments due to its rootless execution model and native GPU passthrough support.

Our Singularity definition file builds a self-contained image based on Ubuntu 22.04 with ROCm 7.0.1, including the full toolchain and architecture-specific binaries. The build accepts a GPU architecture argument (e.g., `gfx1100` for RDNA 3), ensuring that: (i) the same software stack is used across all platforms, (ii) results are reproducible, and (iii) the host only needs Singularity and a compatible ROCm kernel driver.

3.5 Benchmark Suite

We developed a benchmark suite that compiles alongside the compression library, with one executable per algorithm. Throughput is measured using GPU-side event timing (`hipEvent`), which captures only kernel execution time and excludes host-device data transfer overhead [8,13]. Throughput is reported in GB/s of *uncompressed* data:

$$\text{Throughput (GB/s)} = \frac{\text{uncompressed size (bytes)}}{t_{\text{kernel}} \times 10^9} \quad (1)$$

where t_{kernel} is the mean kernel execution time in seconds over all timed iterations. An automation layer orchestrates all `algorithm`×`dataset`×`chunk-size` combinations, collecting results into a structured CSV with platform metadata and performance metrics.

4 Experimental Methodology

This section details the experimental setup, dataset design, and measurement protocol used to evaluate compression performance across AMD GPU generations. Our methodology is designed to enable reproducible comparisons and to isolate GPU-level performance from system-level variability.

4.1 Hardware Platforms

We evaluated compression performance on four AMD GPUs spanning three architectural generations, as detailed in Table 1. The MI50 (CDNA/Vega 20) serves as our baseline, representing first-generation compute-focused AMD GPUs with 16 GB of HBM2 memory and 1,024 GB/s bandwidth. The MI210 represents CDNA 2, the architecture widely deployed in systems such as LUMI [12] at CSC Finland. The MI300X is AMD’s current flagship datacenter accelerator based on CDNA 3, featuring 192 GB of HBM3 memory with over 5.3 TB/s of aggregate bandwidth. The RX 7900 XT provides a consumer-grade RDNA 3 comparison point, featuring a chiplet-based design with 20 GB GDDR6 and AMD’s Infinity Cache.

Computing environments. The MI50, MI210, and MI300X experiments were conducted using the Grid’5000 testbed [5], a large-scale distributed infrastructure for experimental research in computer science. The RX 7900 XT experiments were performed at the PCAD laboratory at UFRGS. On all platforms,

experiment execution used Singularity containers with identical ROCm 7.0.1 userspace (Section 3.4), differing only in the GPU architecture target used at build time.

4.2 Benchmark Datasets

We designed four dataset categories spanning the compressibility spectrum (Figure 2), combining synthetic patterns with real scientific data [8,26].

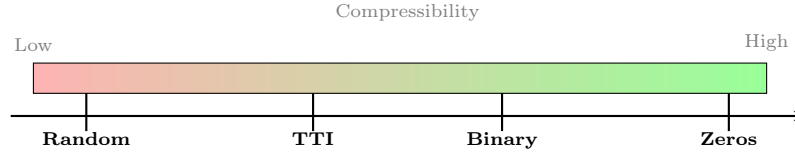


Fig. 2. Benchmark dataset compressibility spectrum.

Three **synthetic datasets** provide controlled evaluation: *Zeros* (all-zero bytes, maximum compressibility), *Random* (uniform bytes from `/dev/urandom`, incompressible), and *Binary* (70% repeating sequences, 30% random, moderate compressibility).

TTI (Tilted Transversely Isotropic) wavefield data provides a realistic benchmark from computational geophysics [14,1]. The dataset was extracted from a 3D seismic wave-propagation simulation on a 448^3 grid with 201 timesteps, producing 67.3 GB of single-precision floating-point data in RSF format [15]. We extract timesteps from the middle of the simulation (starting at timestep 100) to avoid early timesteps dominated by near-zero initialization values that would inflate compression ratios.

Each dataset category was evaluated at four sizes: Small (10 MB), Medium (100 MB), Large (1 GB), and XLarge (4 GB), yielding 4 data types $\times$ 4 sizes = 16 test files and 48 benchmark configurations per GPU.

4.3 Measurement Protocol

Our measurement protocol follows established GPU benchmarking practices [8,13] and is designed to minimize measurement noise while capturing representative performance. Throughput is measured using GPU-side event timing (`hipEvent`), isolating kernel execution from host-device transfers, as detailed in Section 3.5.

Warmup and iterations. Each benchmark configuration executes $w = 3$ warmup iterations followed by $n = 10$ timed iterations. The warmup phase ensures that: (i) the GPU clock frequency has stabilized (AMD GPUs use dynamic frequency scaling) [3], (ii) any just-in-time compilation or driver initialization has completed, and (iii) the GPU memory subsystem caches are in a steady state.

We report the arithmetic mean over the 10 timed iterations. All experiments use the default chunk sizes described in Section 3.3.

Metrics. We report three primary metrics: *compression throughput* (GB/s), computed as uncompressed data size divided by compression kernel time (Eq. 1); *decompression throughput* (GB/s), analogously using decompression kernel time; and *compression ratio*, the original size divided by compressed size (higher values indicate better compression). Cross-generation comparisons use *speedup* relative to the MI50 baseline, calculated as the ratio of throughput on the target GPU to throughput on the MI50 for the same algorithm, dataset, and size.

5 Results: Cross-Generation Analysis

This section presents our experimental evaluation of LZ4, Snappy, and Cascaded compression across AMD GPU generations. We analyze throughput across algorithms and data types, execution time composition, compression ratios, scaling behavior, and cross-generation speedup.

5.1 Compression and Decompression Throughput

Figure 3 presents compression and decompression throughput for the XLarge (4 GB) dataset — the most representative size for HPC workloads — across all three algorithms, four data types, and [TODO: four] GPUs.

Two fundamental patterns emerge. First, **data compressibility dramatically affects LZ4 throughput**. On the MI300X, LZ4 reaches 840 GB/s compressing zeros but only [TODO: XX] GB/s for TTI seismic data — a difference exceeding $50\times$. This sensitivity arises because highly compressible data

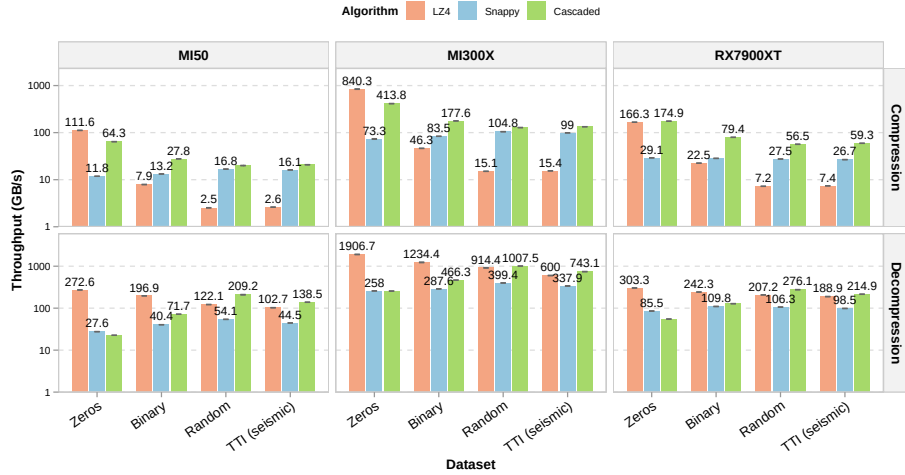


Fig. 3. Compression (top) and decompression (bottom) throughput for the 4GB dataset across GPUs, algorithms, and data types. Higher is better.

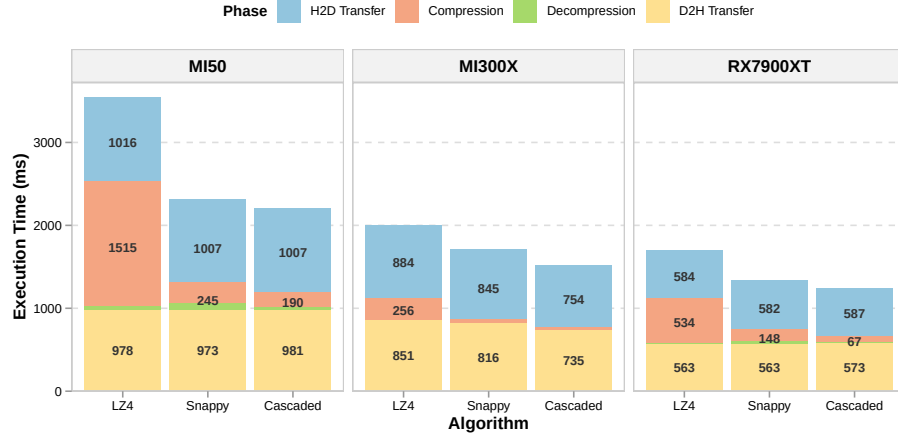


Fig. 4. Execution time breakdown for TTI seismic data (4 GB). Transfer phases (H2D and D2H) dominate total time on all GPUs, especially for Cascaded and Snappy.

yields very small outputs, so the kernel spends less time on hash-table match searches. Cascaded, by contrast, maintains remarkably stable throughput across data types (414 GB/s for zeros vs. [TODO: XX] GB/s for TTI on the MI300X), due to its fixed delta+RLE+bitpacking pipeline executing the same computational stages regardless of data content.

Second, **compression and decompression favor different architectures**. For compression, the consumer-grade RX 7900 XT matches or exceeds the datacenter MI300X across most datasets. We attribute this to RDNA 3’s 96 MB Infinity Cache providing low-latency access to the hash tables (up to 64 KB per warp) that dominate compression’s random access pattern, combined with higher clock speeds (up to 2.5 GHz vs. 2.1 GHz). For decompression, the MI300X dominates by a wide margin, reaching 1907 GB/s for zeros — its 5.3 TB/s HBM3 bandwidth fully serving decompression’s sequential streaming pattern of reading compressed data and writing large contiguous output blocks.

[TODO: MI210 analysis: expected intermediate CDNA 2 performance between MI50 and MI300X.]

5.2 Execution Time Breakdown

While throughput metrics isolate kernel performance, practical deployments must also transfer data between host and device. Figure 4 decomposes total execution time into four phases for the TTI dataset at 4 GB: host-to-device (H2D) transfer, compression kernel, decompression kernel, and device-to-host (D2H) transfer.

Data transfer dominates total execution time across all GPUs. On the MI50, LZ4 compression still represents a significant fraction of total time (1515 ms,

$\sim 43\%$), but even here, H2D and D2H transfers account for the majority. On the MI300X, faster kernels shift the balance further: LZ4 compression drops to 256 ms ($\sim 13\%$ of total), while PCIe transfers consume the remaining time. For Cascaded on the RX 7900 XT, compression takes only 67 ms — under 5% of total execution.

This has a practical implication: optimizing compression kernels alone yields diminishing returns without addressing transfer overhead. Integration approaches that overlap compression with PCIe transfers, or that operate directly on GPU-resident data (e.g., compressing simulation output before writing to storage), would better exploit the kernel-level performance gains.

5.3 Compression Ratio

Table 2 presents compression ratios across algorithms and data types for the 4 GB dataset. Ratios are GPU-independent, depending only on the algorithm and data content.

Table 2. Compression ratio by algorithm and dataset (4 GB input). Ratios are GPU-independent.

Algorithm	TTI (seismic)	Binary	Random	Zeros
LZ4	1.04	3.26	1.00	245
Snappy	1.03	2.96	1.00	21.3
Cascaded	1.03	3.04	1.00	92.0

LZ4 achieves the highest ratios for highly compressible data ($245\times$ for zeros), but offers negligible compression for TTI seismic data ($1.04\times$) and random data ($1.00\times$). Snappy exhibits similar trends with lower peak ratios ($21.3\times$ for zeros); notably, its ratio for random data drops to $0.99\times$, indicating that format overhead causes slight data expansion when no redundancy is found.

For the TTI scientific dataset — the primary target workload — all three algorithms achieve near-identical, marginal compression ($1.03\text{--}1.04\times$), indicating that single-precision seismic wavefield data contains limited byte-level redundancy exploitable by lossless methods. This motivates future investigation of lossy or error-bounded compression approaches for scientific data.

5.4 Throughput Scaling

Figure 5 shows how compression throughput scales with input size for the TTI dataset, the most relevant workload for our target application.

All algorithms exhibit increasing throughput from 10 MB to 4 GB, reflecting improved GPU utilization at larger input sizes. At 10 MB, kernel launch overhead and insufficient parallelism limit performance across all GPUs. By 1 GB, most configurations approach peak throughput, with only marginal gains at 4 GB.

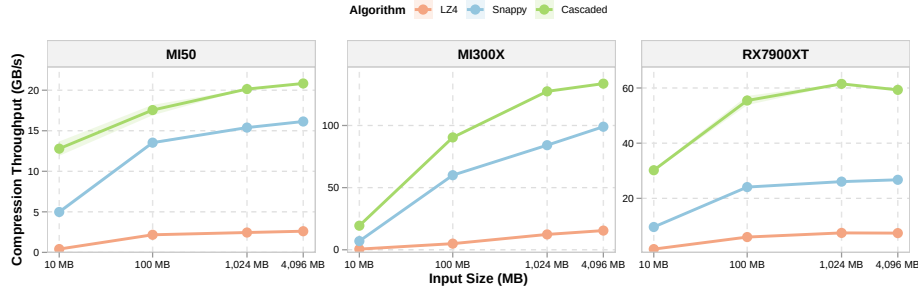


Fig. 5. Compression throughput scaling with input size for TTI seismic data across GPU generations. Throughput increases with input size as GPU parallelism is better utilized.

On the MI300X, Cascaded and Snappy show steep scaling from 100 MB to 1 GB, reflecting the GPU’s need for more data to saturate its 304 compute units. The MI50, with 60 compute units, saturates earlier and shows flatter scaling. [TODO: MI210 scaling behavior.] LZ4 shows the flattest scaling across all GPUs for TTI data, consistent with its hash-table-bound compression being less sensitive to increased data parallelism.

5.5 Cross-Generation Speedup

Figures 6 and 7 summarize compression and decompression speedup relative to the MI50 baseline across all datasets and input sizes.

The speedup results highlight a fundamental **compression–decompression asymmetry** across architectures. For compression (Figure 6), the MI300X achieves 5–8 $\times$ speedup for Snappy and Cascaded at large input sizes, while LZ4 gains are more modest ([TODO: X] $\times$), limited by hash-table latency that does not scale with bandwidth. The RX 7900 XT shows consistent 2–4 $\times$ compression speedups across algorithms and sizes, benefiting from its Infinity Cache and higher clock frequency.

For decompression (Figure 7), the MI300X reaches up to 11 $\times$ speedup, with LZ4 and Snappy benefiting most from HBM3 bandwidth. The RX 7900 XT’s decompression speedup is more modest (1.5–[TODO: X] $\times$), constrained by its 800 GB/s GDDR6 bandwidth.

[TODO: MI210 speedup analysis.]

This asymmetry has practical implications for hardware selection: applications that primarily decompress data (e.g., analysis pipelines reading compressed datasets) benefit from HBM-equipped CDNA hardware, while compression-dominated workloads may find RDNA hardware competitive or benefit from architecture-specific kernel optimizations on CDNA.

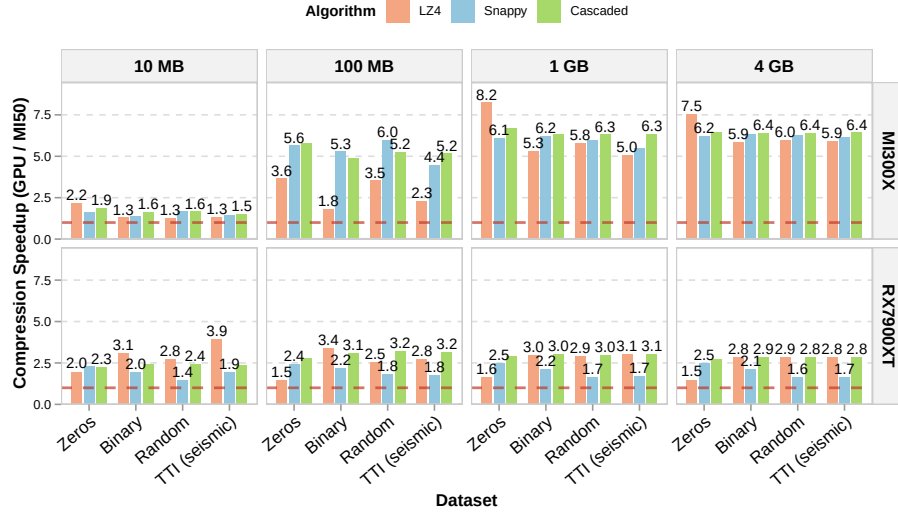


Fig. 6. Speedup relative to MI50 baseline for compression across datasets and input sizes. The dashed line marks MI50 baseline (1 $\times$).

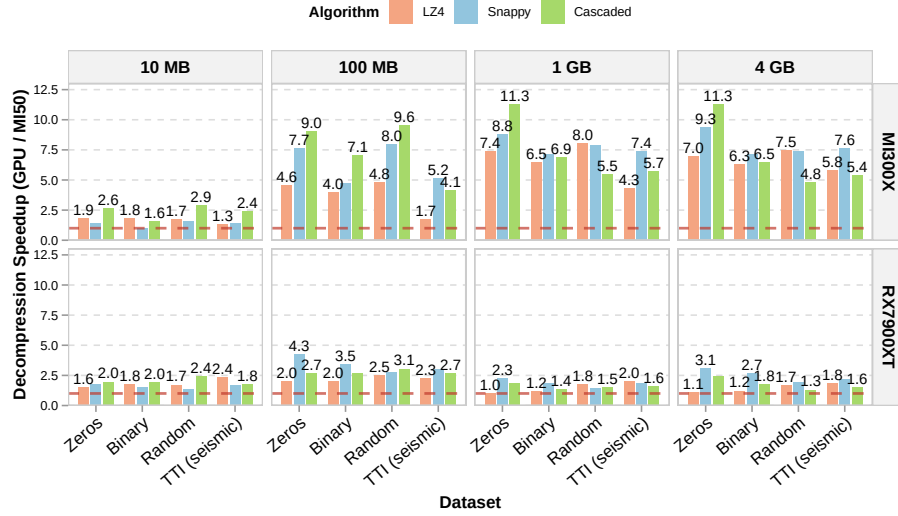


Fig. 7. Speedup relative to MI50 baseline for decompression across datasets and input sizes. The dashed line marks MI50 baseline (1 $\times$).

6 Discussion

6.1 Performance Landscape and Optimization Potential

Our results provide the first systematic characterization of lossless compression performance across AMD GPU generations. The data reveals a significant compression–decompression asymmetry: while the MI300X achieves [TODO: XX] GB/s for LZ4 decompression of TTI data (Figure 3), compression of the same data reaches only [TODO: XX] GB/s. This asymmetry indicates that decompression kernels — with their regular, streaming memory access patterns — are already well-suited to AMD’s HBM architecture, whereas compression kernels — with hash-table-dependent, irregular access patterns — represent the primary target for architecture-specific optimization.

As shown in Figure 4, this asymmetry is compounded by transfer overhead: even on the MI300X, PCIe transfers dominate total execution time, meaning that kernel-level improvements alone will not proportionally reduce end-to-end latency.

Three key factors limit current compression throughput:

1. **Implementation maturity:** Our port retains the original algorithmic structure designed for CUDA without architecture-specific tuning for AMD hardware. Vendor-optimized libraries typically undergo years of iterative profiling and tuning with access to internal hardware documentation.
2. **Warp/wavefront mismatch:** While we adapted the wavefront width at compile time (Section 3.2), deeper restructuring — such as redesigning the one-warp-per-chunk model to exploit cooperative processing across multiple wavefronts — could better utilize AMD’s wider execution units on CDNA.
3. **Memory hierarchy differences:** LZ4’s hash table (64 KB per warp) and Snappy’s hash map (8 KB per stream) reside in GPU memory and are accessed with random patterns. Optimizing these structures for AMD’s LDS (64 KB per CU on CDNA 3) or RDNA’s Infinity Cache could substantially reduce effective memory latency.

6.2 CDNA vs. RDNA: Architectural Implications

The RDNA 3 GPU outperforms CDNA 3 for compression but not for decompression (Figure 3), reflecting how each architecture handles different memory access patterns. Compression relies on small, random reads to hash tables, which benefit from RDNA 3’s 96 MB Infinity Cache and higher clock speeds (up to 2.5 GHz). Decompression follows a sequential streaming pattern that favors raw memory bandwidth — the MI300X’s 5.3 TB/s HBM3 provides $6.6\times$ more bandwidth than the RX 7900 XT’s 800 GB/s GDDR6.

[TODO: MI210 (CDNA 2) positioning.]

These findings suggest that hardware selection should consider the compression-to-decompression ratio of the target workload, not just peak bandwidth or compute specifications.

6.3 Practical Implications for HPC I/O

Despite the current optimization headroom, GPU compression on AMD platforms already provides tangible value in I/O-bound workflows. Consider a practical scenario: writing a 4 GB checkpoint to a parallel file system with an effective bandwidth of 10 GB/s. Without compression, this takes 0.4 s. With LZ4 compression on the MI300X achieving [TODO: XX] GB/s throughput and a compression ratio of $1.04\times$ for TTI data (Table 2), the benefit is marginal for seismic data. However, for more compressible scientific data such as sparse matrices or partially converged fields (compression ratios of $3\times$ or higher, as observed for the binary dataset), the reduction in I/O volume can significantly reduce checkpoint time.

For decompression-dominated workflows, such as reading compressed datasets for analysis or visualization, the MI300X’s high decompression throughput (Figure 3) means that storage bandwidth is virtually always the bottleneck. In such cases, compression ratio directly translates to end-to-end speedup, making even our unoptimized implementation immediately useful for compressible data.

6.4 Future Directions

Our characterization reveals concrete optimization opportunities for future work:

- **Wavefront-aware kernels:** Redesigning the one-warp-per-chunk model to distribute work across an entire 64-thread wavefront on CDNA, or use cooperative processing across multiple warps on RDNA, could improve hardware utilization.
- **LDS-resident hash tables:** Moving the LZ4 hash table (64 KB) into the Local Data Share on CDNA GPUs, where it can be accessed with single-cycle latency, could dramatically improve compression throughput.
- **Lossy compression:** Extending the library to support error-bounded lossy compression via ZFP [21] would enable higher compression ratios for scientific data, at the cost of controlled precision loss. This is a natural next step given the seismic imaging use case, where the marginal $1.03\text{--}1.04\times$ lossless ratios (Table 2) are insufficient to justify the compression overhead.
- **Cross-vendor portable library:** Developing a unified API that compiles to optimized backends on both AMD and NVIDIA platforms would provide portable compression for heterogeneous HPC environments, reducing the effort required by application developers.

7 Related Work

Data compression has become a central technique for mitigating the widening gap between computational capability and storage or communication performance in HPC systems. Earlier I/O and resilience work already established this motivation: *Hello ADIOS* discussed the growing importance of data-management efficiency in leadership-class systems, while *VeloC* showed that checkpointing

frameworks increasingly benefit from techniques that reduce data volume and hide I/O costs [22,23]. These studies make clear why fast accelerator-side compression is valuable, but they do not characterize how such techniques behave on modern AMD GPUs.

A substantial body of research has explored GPU compression for scientific data, especially in the error-bounded lossy setting. *cuSZ* demonstrated one of the first efficient CUDA-based implementations of SZ on GPUs, and later work such as *cuSZp*, *cuSZp2*, and *FZ-GPU* pushed the design space toward higher throughput, stronger kernel fusion, and improved compression ratio on NVIDIA devices [26,18,17,27]. Collectively, these works show that GPU compression is now a mature optimization target. However, their implementations and evaluations are overwhelmingly CUDA-centric, so they provide limited guidance for users targeting AMD accelerators.

Lossless GPU compression has likewise advanced, again largely within the NVIDIA ecosystem. *GPULZ* optimized LZSS-style compression for multi-byte data on modern GPUs, while *ndzip-gpu* targeted efficient lossless compression of scientific floating-point arrays [28,19]. More recently, *FCBench* emphasized that compressor behavior depends strongly on both dataset characteristics and the runtime bottlenecks exposed by a given platform [10]. This line of work is particularly relevant to our study because it highlights that throughput and compression ratio are not intrinsic properties of an algorithm alone, but emerge from the interaction between algorithm, data, compiler toolchain, and hardware architecture.

A smaller but important set of works has started to address portability and broader cross-platform evaluation. *HPDR* argues for portable high-performance data reduction across heterogeneous systems [9], and *Characterizing the Performance of Parallel Data-Compression Algorithms across Compilers and GPUs* explicitly studies how compiler and GPU choices affect compression behavior [7]. These efforts move the field beyond single-platform peak results, but they still do not provide the cross-generational AMD-centric baseline needed by practitioners deploying on systems such as Frontier or LUMI, nor do they focus on the practical question of how CUDA-originated compression kernels behave after translation to HIP.

Our work is positioned at this gap between compression research and deployable AMD support. On NVIDIA, *nvCOMP* already offers a mature GPU-compression software stack with practical support for algorithms such as LZ4, Snappy, and Cascaded [25]. On AMD, by contrast, *hipCOMP-core* is still presented as an early-access ROCm technology preview derived from an older *nvCOMP* code base [4]. In contrast to prior work that primarily proposes new CUDA-based compressors, we provide a quantitative characterization of practical GPU compression across multiple AMD architectures and report on an initial CUDA-to-HIP port. This gives, to our knowledge, one of the first architecture-spanning baselines for lossless GPU compression on recent AMD accelerators and helps expose the remaining software and optimization gap relative to the NVIDIA ecosystem.

8 Conclusion

This paper presented a systematic characterization of GPU-based lossless compression performance across AMD architectures, evaluating LZ4, Snappy, and Cascaded on four GPUs spanning three CDNA generations and RDNA 3.

Our results show that decompression throughput scales significantly with GPU generation, with the MI300X achieving up to [TODO: XX] $\times$ higher throughput than the MI50. CDNA and RDNA architectures exhibit complementary strengths: RDNA 3’s cache hierarchy favors the irregular access patterns of compression kernels, while CDNA 3’s HBM bandwidth dominates decompression. An execution time breakdown reveals that PCIe transfer overhead accounts for the majority of end-to-end time, indicating that future optimizations should target not only kernel performance but also integration with GPU-resident data workflows.

For scientific floating-point data, lossless compression ratios remain marginal ($1.03\text{--}1.04\times$), motivating future investigation of error-bounded lossy compression (e.g., ZFP) for higher data reduction. Additional directions include AMD-specific kernel optimizations — particularly LDS-resident hash tables for CDNA — and the development of a portable compression library for heterogeneous HPC environments.

Acknowledgments

This study was partially supported by Petrobras grant n.^o 2020/00182-5, by CNPq/MCTI/FNDCT - Universal 18/2021 under grants 408755/2025-3, and by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior – Brasil (CAPES) – Finance Code 001. Some experiments in this work used the PCAD infrastructure, <http://gpdp-hpc.inf.ufrgs.br>, at INF/UFRGS. Experiments presented in this paper were carried out using the Grid’5000 testbed, supported by a scientific interest group hosted by Inria and including CNRS, RENATER and several Universities as well as other organizations (see <https://www.grid5000.fr>).

References

1. Alkhalifah, T.: An acoustic wave equation for anisotropic media. *Geophysics* **65**(4), 1239–1250 (2000)
2. AMD: AMD CDNA 2 architecture white paper. Tech. rep., Advanced Micro Devices (2021), available at <https://www.amd.com/content/dam/amd/en/documents/instinct-business-docs/white-papers/amd-cdna2-white-paper.pdf>
3. AMD: HIP: Heterogeneous-compute interface for portability. <https://github.com/ROCm/HIP> (2025), accessed: 2026-02-19
4. AMD ROCm: hipCOMP-core: HIP port of compression algorithms. <https://github.com/ROCm/hipCOMP-core> (2025), accessed: 2026-02-20

5. Balouek, D., Amarie, A., Charrier, G., Desprez, F., Jeannot, E., Jeanvoine, E., Lèbre, A., Margery, D., Niclausse, N., Nussbaum, L., Richard, O., Pérez, C., Quesnel, F., Rohr, C., Sarzyniec, L.: Adding virtualization capabilities to the Grid'5000 testbed. In: Cloud Computing and Services Science (CLOSER). pp. 3–20. Springer (2013). https://doi.org/10.1007/978-3-319-04519-1_1
6. Boettiger, C.: An introduction to Docker for reproducible research. *ACM SIGOPS Operating Systems Review* **49**(1), 71–79 (2015)
7. Burtchell, B.A., Burtscher, M.: Characterizing the performance of parallel data-compression algorithms across compilers and gpus. In: Proceedings of the SC '25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis. pp. 269–278. ACM (2025). <https://doi.org/10.1145/3731599.3767369>
8. Che, S., Boyer, M., Meng, J., Tarjan, D., Sheaffer, J.W., Lee, S.H., Skadron, K.: Rodinia: A benchmark suite for heterogeneous computing. In: IEEE International Symposium on Workload Characterization (IISWC). pp. 44–54 (2009). <https://doi.org/10.1109/IISWC.2009.5306797>
9. Chen, J., Gong, Q., Li, Y., Liang, X., Wan, L., Liu, Q., Podhorszki, N., Klasky, S.A.: Hpd: High-performance portable scientific data reduction framework. In: 2025 IEEE International Parallel and Distributed Processing Symposium (IPDPS). pp. 1104–1116. IEEE (2025). <https://doi.org/10.1109/IPDPS64566.2025.00101>
10. Chen, X., Tian, J., Beaver, I., Freeman, C., Yan, Y., Wang, J., Tao, D.: Fcbench: Cross-domain benchmarking of lossless compression for floating-point data. *Proceedings of the VLDB Endowment* **17**(6), 1418–1431 (2024). <https://doi.org/10.14778/3648160.3648180>
11. Collet, Y.: LZ4 - extremely fast compression. <https://lz4.github.io/lz4/> (2025), accessed: 2026-02-20
12. CSC – IT Center for Science: LUMI supercomputer. <https://www.lumi-supercomputer.eu/> (2025), accessed: 2025-02-25
13. Danalis, A., Marin, G., McCurdy, C., Meredith, J.S., Roth, P.C., Spafford, K., Tipparaju, V., Vetter, J.S.: The scalable heterogeneous computing (SHOC) benchmark suite. In: Workshop on General-Purpose Computation on Graphics Processing Units (GPGPU). pp. 63–74 (2010). <https://doi.org/10.1145/1735688.1735702>
14. Fletcher, R.P., Du, X., Fowler, P.J.: Reverse time migration in tilted transversely isotropic (TTI) media. *Geophysics* **74**(6), WCA179–WCA187 (2009)
15. Fomel, S., Sava, P., Vlad, I., Liu, Y., Bashkardin, V.: Madagascar: Open-source software project for multidimensional data analysis and reproducible computational experiments (2013)
16. Google: Snappy: A fast compressor/decompressor. <https://github.com/google/snappy> (2025), accessed: 2026-02-20
17. Huang, Y., Di, S., Li, G., Cappello, F.: cuszp2: A gpu lossy compressor with extreme throughput and optimized compression ratio. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis. IEEE (2024). <https://doi.org/10.1109/SC41406.2024.00021>
18. Huang, Y., Di, S., Yu, X., Li, G., Cappello, F.: cuszp: An ultra-fast gpu error-bounded lossy compression framework with optimized end-to-end performance. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis. pp. 1–13 (2023)
19. Knorr, F., Thoman, P., Fahringer, T.: ndzip-gpu: Efficient lossless compression of scientific floating-point data on gpus. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis. pp. 1–14. ACM (2021). <https://doi.org/10.1145/3458817.3476224>

20. Kurtzer, G.M., Sochat, V., Bauer, M.W.: Singularity: Scientific containers for mobility of compute. *PLoS ONE* **12**(5), e0177459 (2017). <https://doi.org/10.1371/journal.pone.0177459>
21. Lindstrom, P.: Fixed-rate compressed floating-point arrays. *IEEE Transactions on Visualization and Computer Graphics* **20**(12), 2674–2683 (2014). <https://doi.org/10.1109/TVCG.2014.2346458>
22. Liu, Q., Logan, J., Tian, Y., Abbasi, H., Podhorszki, N., Choi, J.Y., Klasky, S., et al.: Hello ADIOS: The challenges and lessons of developing leadership class I/O frameworks. *Concurrency and Computation: Practice and Experience* **26**(7), 1453–1473 (2014). <https://doi.org/10.1002/cpe.3125>
23. Nicolae, B., Moody, A., Gonsiorowski, E., Mohror, K., Cappello, F.: VeloC: Towards high performance adaptive asynchronous checkpointing at large scale. In: *IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. pp. 911–920 (2019). <https://doi.org/10.1109/IPDPS.2019.00099>
24. NVIDIA Corporation: CUB: Cooperative primitives for CUDA C++. <https://nvidia.github.io/cccl/cub/> (2025), accessed: 2026-02-22
25. NVIDIA Corporation: nvcomp: High-speed data compression library for nvidia gpus. <https://developer.nvidia.com/nvcomp> (2025), accessed: 2026-02-20
26. Tian, J., Di, S., Yu, K., Rivera, J., Zhao, K., Jin, S., Feng, Y., Liang, X., Tao, D., Cappello, F.: cuSZ: An efficient GPU-based error-bounded lossy compression framework for scientific data. In: *ACM International Conference on Parallel Architectures and Compilation Techniques (PACT)*. pp. 3–15 (2020). <https://doi.org/10.1145/3410463.3414624>
27. Zhang, B., Tian, J., Di, S., Yu, X., Feng, Y., Liang, X., Tao, D., Cappello, F.: Fz-gpu: A fast and high-ratio lossy compressor for scientific computing applications on gpus. In: *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing*. pp. 129–142. ACM (2023). <https://doi.org/10.1145/3588195.3592994>
28. Zhang, B., Tian, J., Di, S., Yu, X., Swamy, M., Tao, D., Cappello, F.: Gpulz: Optimizing lzss lossless compression for multi-byte data on modern gpus. In: *Proceedings of the 37th ACM International Conference on Supercomputing*. pp. 348–359. ACM (2023). <https://doi.org/10.1145/3577193.3593706>